Topico 4: Capacidade de Canal e Codigos Corretores de Erro

Cenario: link de radio Terra-Marte (canal binario simetrico). Enunciado: `propostas/1_Simulacoes.pdf`, secao 4.

Preparacao

In [1]:
```
import sys, pathlib

REPO = pathlib.Path.cwd().parents[2]          # .../<topico>/ -> .../<proposta>/ -> raiz
if str(REPO) not in sys.path:
    sys.path.insert(0, str(REPO))

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from PIL import Image
from scipy.stats import binom

from utils.info_theory import (
    entropy, binary_entropy, joint_entropy, conditional_entropy,
    cross_entropy, kl_divergence, mutual_information,
    conditional_mutual_information, mi_gtest_statistic,
)
from utils.simulation import (
    rng, sample_pmf, sample_joint, empirical_pmf, bsc, markov_chain,
)
from utils.estimation import bootstrap_ci
from utils.plotting import setup_estilo, salvar_fig, plot_curvas, PALETA, salvar_tabela
import src.model as model                     # helpers especificos deste topico

setup_estilo()
GEN = rng(20260902)                           # seed fixo -> resultados reproduziveis
DADOS = pathlib.Path("dados")
FIGURAS = pathlib.Path("figuras")
ENTRADA = pathlib.Path("entrada")             # ativos de entrada versionados (ex.: sample.jpg)
```

Passo 1: Canal BSC

Modelar o canal com parametro p (algo entre 0.05 e 0.2).

In [2]:
```
# Passo 1
P_CANAL = 0.1  # probabilidade de erro por uso do canal (link Terra-Marte ruidoso)

print(f"p do canal binario simetrico = {P_CANAL}")
```

```
p do canal binario simetrico = 0.1
```

Passo 2: Capacidade teorica

C = 1 - binary_entropy(p), em bits por uso do canal.

In [3]:
```
# Passo 2
H_B_P = binary_entropy(P_CANAL)
C = 1 - H_B_P

print(f"H_b(p) = {H_B_P:.4f} bits")
print(f"capacidade C = 1 - H_b(p) = {C:.4f} bits por uso do canal")
```

```
H_b(p) = 0.4690 bits
capacidade C = 1 - H_b(p) = 0.5310 bits por uso do canal
```

Passo 3: Mensagem

Se houver uma imagem em `entrada/sample.jpg` (versionada, diferente de `dados/` / `figuras/`), carrega-la, aplicar um filtro de binarizacao (limiar sobre a escala de cinza) e usar os pixels binarios como sequencia de bits de informacao -- imagem de verdade em vez da mensagem sintetica, bem mais bits, util para checar a performance dos passos seguintes em escala maior. Sem essa imagem (caso raro, ex.: `entrada/` removida localmente), o passo cai de volta para a mensagem sintetica original.

In [4]:
```
# Passo 3
CAMINHO_IMAGEM = ENTRADA / "sample.jpg"


def binariza_por_limiar(imagem_cinza: np.ndarray, limiar: float) -> np.ndarray:
    """Filtro de binarizacao: cada pixel vira 1 se estiver acima do limiar."""
    return (imagem_cinza > limiar).astype(np.uint8)


if CAMINHO_IMAGEM.exists():
    IMAGEM_CINZA = np.asarray(Image.open(CAMINHO_IMAGEM).convert("L"))
    LIMIAR = IMAGEM_CINZA.mean()  # limiar simples: media de cinza da propria imagem
    IMAGEM_BINARIA = binariza_por_limiar(IMAGEM_CINZA, LIMIAR)

    fig, eixos = plt.subplots(1, 2, figsize=(10, 5))
    eixos[0].imshow(IMAGEM_CINZA, cmap="gray")
```

```
    eixos[0].set_title("original (escala de cinza)")
    eixos[0].axis("off")
    eixos[1].imshow(IMAGEM_BINARIA, cmap="gray")
    eixos[1].set_title(f"binarizada (limiar = {LIMIAR:.1f})")
    eixos[1].axis("off")
    salvar_fig("t4_passo3_imagem_binarizada")
    plt.show()

    MSG_COMPLETA = IMAGEM_BINARIA.ravel()
    N_BITS = MSG_COMPLETA.size - (MSG_COMPLETA.size % 4)  # Hamming(7,4)
exige multiplo de 4
    MSG = MSG_COMPLETA[:N_BITS]
    SOBRA_IMAGEM = MSG_COMPLETA[N_BITS:]  # poucos bits cortados; guardados
para reconstruir a imagem no Passo 10
    print(f"mensagem: imagem real
({IMAGEM_CINZA.shape[1]}x{IMAGEM_CINZA.shape[0]})")
else:
    # sem imagem local (caso raro) -> mensagem sintetica de sempre
    N_BITS = 200_000  # pixels de uma imagem simplificada, preto/branco
equiprovaveis
    MSG = sample_pmf([0.5, 0.5], size=N_BITS,
generator=GEN).astype(np.uint8)
    print("mensagem: sintetica (entrada/sample.jpg nao encontrada)")

print(f"N_BITS = {N_BITS:,}")
print(f"primeiros 20 bits: {MSG[:20]}")
print(f"fracao de 1s na mensagem: {MSG.mean():.4f}")
```




```
mensagem: imagem real (2570x3427)
N_BITS = 8,807,388
primeiros 20 bits: [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
fracao de 1s na mensagem: 0.5204
```

Passo 4: Transmissao direta

Passar os bits pelo `bsc` sem codificacao; medir a BER e comparar com p.

In [5]:
```
# Passo 4
RECEBIDO_DIRETO = bsc(MSG, P_CANAL, generator=GEN)
ber_direto = np.mean(RECEBIDO_DIRETO != MSG)

print(f"BER sem codificacao: {ber_direto:.4f}")
print(f"p do canal:          {P_CANAL:.4f}")
print(f"diferenca:           {abs(ber_direto - P_CANAL):.4f}")
```

```
BER sem codificacao: 0.1001
p do canal:          0.1000
diferenca:           0.0001
```

Passo 5: Codigo de repeticao

`model.repetition_encode` / `repetition_decode` (n_rep impar); medir a taxa efetiva R = 1/n_rep e a BER pos-decodificacao.

In [6]:
```
# Passo 5
N_REP = 3  # impar -> voto de maioria sem empate

COD_REP = model.repetition_encode(MSG, N_REP)
RECEBIDO_REP = bsc(COD_REP, P_CANAL, generator=GEN)
DECOD_REP = model.repetition_decode(RECEBIDO_REP, N_REP)

R_REP = 1 / N_REP
ber_rep = np.mean(DECOD_REP != MSG)

print(f"n_rep = {N_REP}")
print(f"taxa efetiva R = 1/n_rep = {R_REP:.4f}")
print(f"BER pos-decodificacao (repeticao): {ber_rep:.6f}")
```

```
n_rep = 3
taxa efetiva R = 1/n_rep = 0.3333
BER pos-decodificacao (repeticao): 0.028023
```

Passo 6: Hamming(7,4)

`model.hamming74_encode` / `hamming74_decode` (decodificacao por sindrome); medir R = 4/7 e a BER.

In [7]:
```
# Passo 6
COD_HAM = model.hamming74_encode(MSG)
RECEBIDO_HAM = bsc(COD_HAM, P_CANAL, generator=GEN)
DECOD_HAM = model.hamming74_decode(RECEBIDO_HAM)

R_HAM = 4 / 7
ber_ham = np.mean(DECOD_HAM != MSG)
```

```
print(f"taxa efetiva R = 4/7 = {R_HAM:.4f}")
print(f"BER pos-decodificacao (Hamming(7,4)): {ber_ham:.6f}")
```

```
taxa efetiva R = 4/7 = 0.5714
BER pos-decodificacao (Hamming(7,4)): 0.066832
```

Passo 7: Tabela comparativa

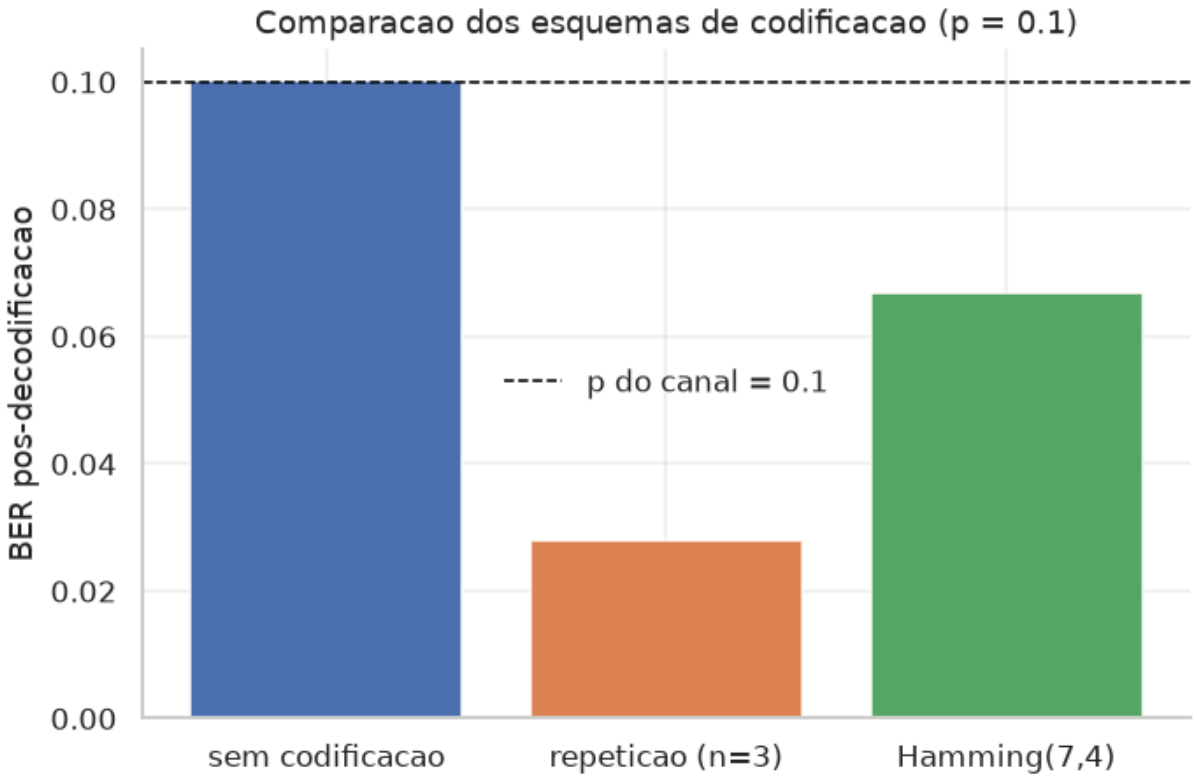
BER e taxa efetiva das tres abordagens para o mesmo p.

In [8]:
```
# Passo 7
TABELA_ESQUEMAS = pd.DataFrame(
    {
        "esquema": ["sem codificacao", f"repeticao (n={N_REP})",
"Hamming(7,4)"],
        "R": [1.0, R_REP, R_HAM],
        "BER": [ber_direto, ber_rep, ber_ham],
    }
).set_index("esquema")

salvar_tabela(TABELA_ESQUEMAS, "t4_passo7_comparacao_esquemas")
display(TABELA_ESQUEMAS)

fig, ax = plt.subplots()
ax.bar(TABELA_ESQUEMAS.index, TABELA_ESQUEMAS["BER"], color=PALETA[:3])
ax.axhline(P_CANAL, ls="--", color="black", lw=1, label=f"p do canal =
{P_CANAL}")
ax.set_ylabel("BER pos-decodificacao")
ax.set_title(f"Comparacao dos esquemas de codificacao (p = {P_CANAL})")
ax.legend()
salvar_fig("t4_passo7_comparacao_esquemas")
plt.show()
```

esquema	R	BER
sem codificacao	1.000000	0.100093
repeticao (n=3)	0.333333	0.028023
Hamming(7,4)	0.571429	0.066832



Passo 8: Variar p

Repetir para varios p e observar como muda a vantagem de cada codigo.

In [9]:
```
# Passo 8
P_VALORES = np.array([0.02, 0.05, 0.1, 0.15, 0.2, 0.3])

registros = []
for p in P_VALORES:
    recebido_direto_p = bsc(MSG, p, generator=GEN)
    ber_direto_p = np.mean(recebido_direto_p != MSG)

    recebido_rep_p = bsc(COD_REP, p, generator=GEN)
    decod_rep_p = model.repetition_decode(recebido_rep_p, N_REP)
    ber_rep_p = np.mean(decod_rep_p != MSG)

    recebido_ham_p = bsc(COD_HAM, p, generator=GEN)
    decod_ham_p = model.hamming74_decode(recebido_ham_p)
    ber_ham_p = np.mean(decod_ham_p != MSG)

    registros.append({"p": p, "esquema": "sem codificacao", "BER":
ber_direto_p})
    registros.append({"p": p, "esquema": f"repeticao (n={N_REP})", "BER":
ber_rep_p})
    registros.append({"p": p, "esquema": "Hamming(7,4)", "BER": ber_ham_p})

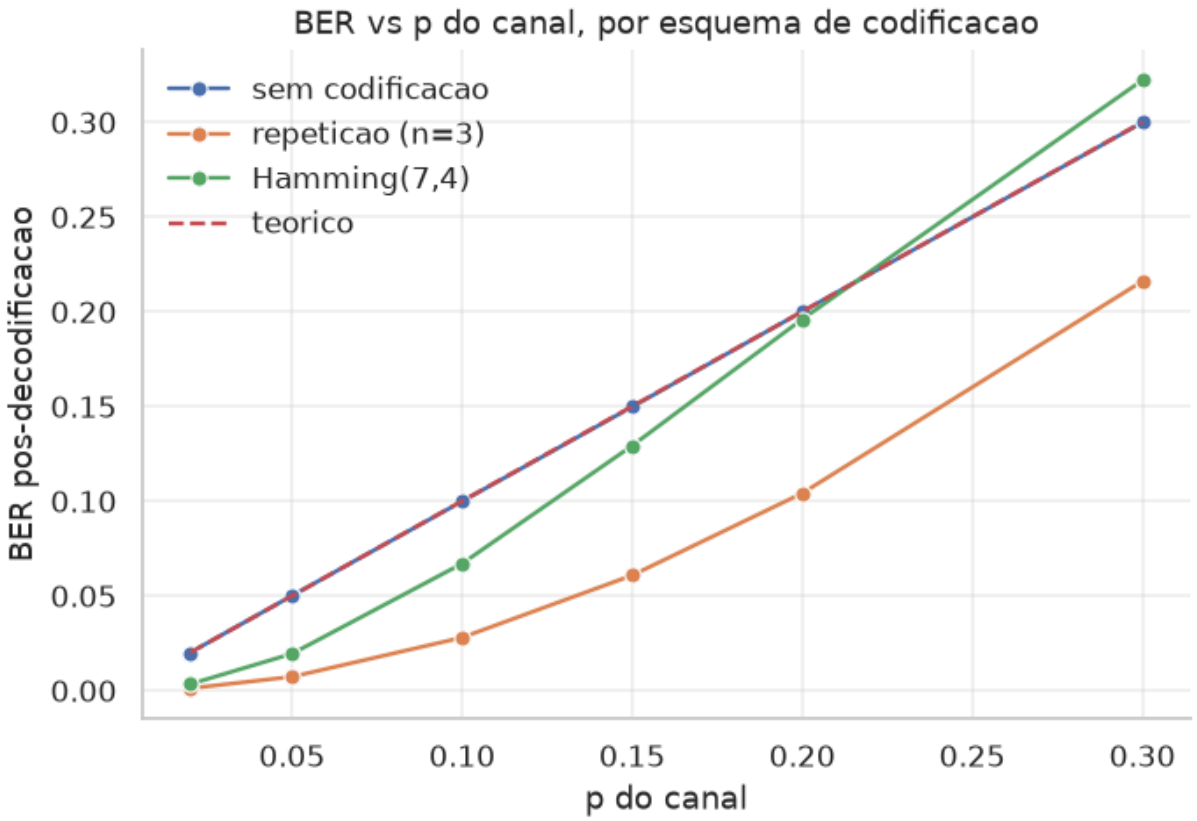
DF_VARIANDO_P = pd.DataFrame(registros)

TABELA_VARIANDO_P = DF_VARIANDO_P.pivot(index="p", columns="esquema",
```

```
values="BER")
salvar_tabela(TABELA_VARIANDO_P, "t4_passo8_ber_vs_p")
display(TABELA_VARIANDO_P)

plot_curvas(
    DF_VARIANDO_P, x="p", y="BER", hue="esquema", teorico=lambda p: p,
    xlabel="p do canal", ylabel="BER pos-decodificacao",
)
plt.title("BER vs p do canal, por esquema de codificacao")
salvar_fig("t4_passo8_ber_vs_p")
plt.show()
```

esquema	Hamming(7,4)	repeticao (n=3)	sem codificacao
p			
0.02	0.003342	0.001164	0.019980
0.05	0.019431	0.007234	0.050074
0.10	0.067010	0.027964	0.099874
0.15	0.129248	0.060816	0.149775
0.20	0.196014	0.104136	0.199982
0.30	0.322186	0.216050	0.299838



Passo 9: Promessa de Shannon

Aumentar n_rep = 3, 5, 7, 9, ...; ver a BER de bloco cair e relacionar com a distancia entre R e C. Conferir com a cauda binomial (`scipy.stats.binom`).

In [10]:

```
# Passo 9
N_REP_VALORES = np.array([3, 5, 7, 9, 11, 13, 15])

registros_n_rep = []
for n_rep in N_REP_VALORES:
    cod_n = model.repetition_encode(MSG, n_rep)
    recebido_n = bsc(cod_n, P_CANAL, generator=GEN)
    decod_n = model.repetition_decode(recebido_n, n_rep)

    r_n = 1 / n_rep
    ber_empirica_n = np.mean(decod_n != MSG)

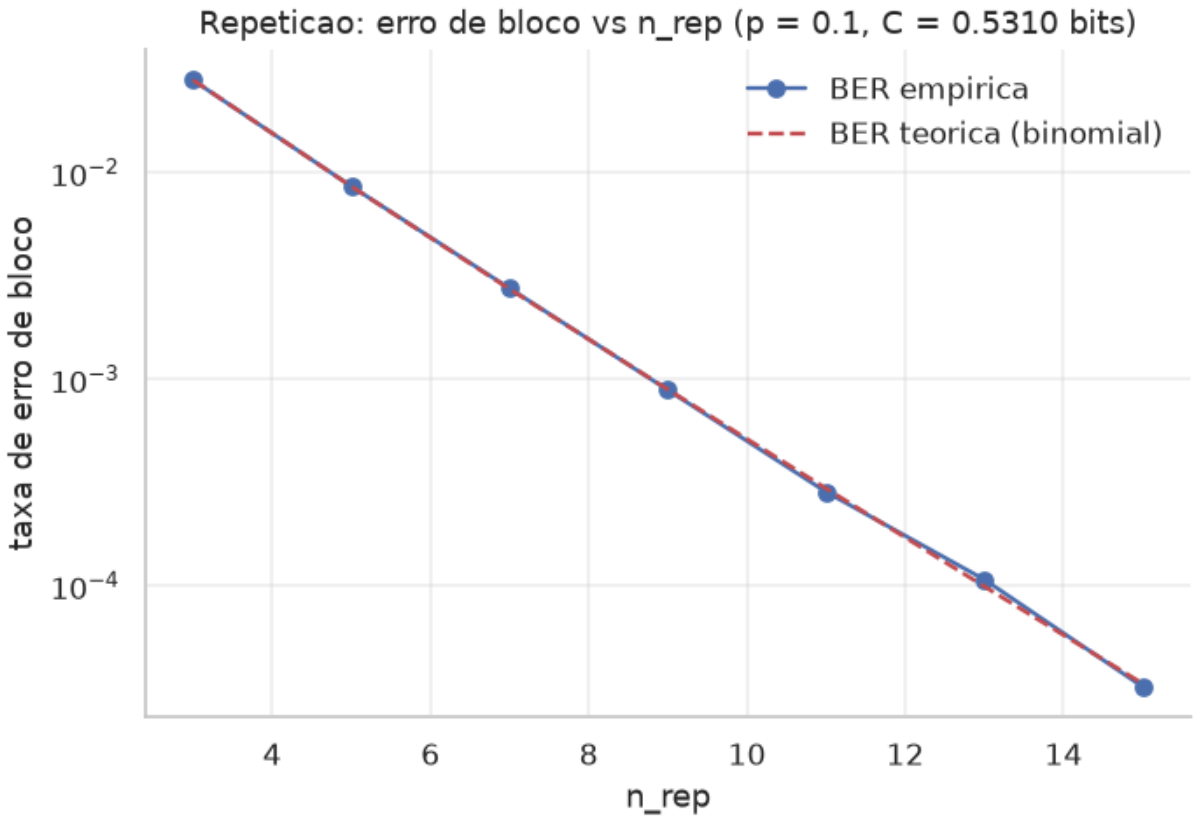
    # decodificacao por maioria falha quando mais da metade dos n_rep bits
do
    # bloco vira: cauda superior da binomial(n_rep, p) a partir da maioria.
    limiar = n_rep // 2
    ber_teorica_n = binom.sf(limiar, n_rep, P_CANAL)

    registros_n_rep.append(
        {
            "n_rep": n_rep,
            "R": r_n,
            "C_menos_R": C - r_n,
            "BER_empirica": ber_empirica_n,
            "BER_teorica": ber_teorica_n,
        }
    )

DF_N_REP = pd.DataFrame(registros_n_rep).set_index("n_rep")
salvar_tabela(DF_N_REP, "t4_passo9_variando_n_rep")
display(DF_N_REP)

fig, ax = plt.subplots()
ax.plot(DF_N_REP.index, DF_N_REP["BER_empirica"], "o-", color=PALETA[0],
label="BER empirica")
ax.plot(DF_N_REP.index, DF_N_REP["BER_teorica"], "--", color=PALETA[3],
label="BER teorica (binomial)")
ax.set_yscale("log")
ax.set_xlabel("n_rep")
ax.set_ylabel("taxa de erro de bloco")
ax.set_title(f"Repeticao: erro de bloco vs n_rep (p = {P_CANAL}, C =
{C:.4f} bits)")
ax.legend()
salvar_fig("t4_passo9_variando_n_rep")
plt.show()
```

n_rep	R	C_menos_R	BER_empirica	BER_teorica
3	0.333333	0.197671	0.027955	0.028000
5	0.200000	0.331004	0.008569	0.008560
7	0.142857	0.388147	0.002748	0.002728
9	0.111111	0.419893	0.000886	0.000891
11	0.090909	0.440095	0.000285	0.000296
13	0.076923	0.454081	0.000107	0.000099
15	0.066667	0.464338	0.000032	0.000034



Passo 10: Cenario R > C

Escolher um canal bem mais ruidoso, com capacidade C bem menor que a taxa efetiva R de qualquer um dos tres esquemas; tentar reduzir a BER a um nivel aceitavel e documentar que nao e possivel. Reconstruir a imagem binarizada original lado a lado com a varredura de n_rep do Passo 9 (R < C, canal original) e os tres esquemas deste passo (R > C, canal ruidoso), para verificar visualmente o resultado final.

In [11]:
```
# Passo 10
P_CANAL_ALTO = 0.4  # canal bem mais ruidoso -> capacidade quase nula
```

```
C_ALTO = 1 - binary_entropy(P_CANAL_ALTO)
BER_ACEITAVEL = 0.01  # limiar arbitrario de "aceitavel" para a discussao

RECEBIDO_DIRETO_ALTO = bsc(MSG, P_CANAL_ALTO, generator=GEN)
ber_direto_alto = np.mean(RECEBIDO_DIRETO_ALTO != MSG)

RECEBIDO_REP_ALTO = bsc(COD_REP, P_CANAL_ALTO, generator=GEN)
DECOD_REP_ALTO = model.repetition_decode(RECEBIDO_REP_ALTO, N_REP)
ber_rep_alto = np.mean(DECOD_REP_ALTO != MSG)

RECEBIDO_HAM_ALTO = bsc(COD_HAM, P_CANAL_ALTO, generator=GEN)
DECOD_HAM_ALTO = model.hamming74_decode(RECEBIDO_HAM_ALTO)
ber_ham_alto = np.mean(DECOD_HAM_ALTO != MSG)

TABELA_R_MAIOR_C = pd.DataFrame(
    {
        "esquema": ["sem codificacao", f"repeticao (n={N_REP})",
"Hamming(7,4)"],
        "R": [1.0, R_REP, R_HAM],
        "BER": [ber_direto_alto, ber_rep_alto, ber_ham_alto],
    }
).set_index("esquema")
TABELA_R_MAIOR_C["R > C"] = TABELA_R_MAIOR_C["R"] > C_ALTO
TABELA_R_MAIOR_C["BER aceitavel"] = TABELA_R_MAIOR_C["BER"] < BER_ACEITAVEL

salvar_tabela(TABELA_R_MAIOR_C, "t4_passo10_r_maior_c")
display(TABELA_R_MAIOR_C)

print(f"p do canal = {P_CANAL_ALTO}, capacidade C = {C_ALTO:.4f} bits")
print(f"nenhum dos tres esquemas chega perto de BER < {BER_ACEITAVEL}
(todos com R > C)")

# reconstrucao visual: só faz sentido se MSG veio da imagem real do Passo
3.
# MSG foi cortada para multiplo de 4 (Hamming); SOBRA_IMAGEM completa de
volta
# o tamanho original antes do reshape.
if "IMAGEM_BINARIA" in globals():
    FORMATO = IMAGEM_BINARIA.shape
    PASSO = 4  # mostra 1 a cada 4 pixels por eixo, so para a figura
renderizar rapido

    def reconstroi(bits_decodificados):
        return np.concatenate([bits_decodificados,
SOBRA_IMAGEM]).reshape(FORMATO)

    imagens = {"original": IMAGEM_BINARIA}

    # repete a varredura do Passo 9 (mesmo canal original, R < C) e
reconstroi
    # a imagem recebida em cada n_rep -- mostra a melhora progressiva.
    for n_rep in N_REP_VALORES:
        n_rep = int(n_rep)
        cod_n = model.repetition_encode(MSG, n_rep)
        recebido_n = bsc(cod_n, P_CANAL, generator=GEN)
        decod_n = model.repetition_decode(recebido_n, n_rep)
```

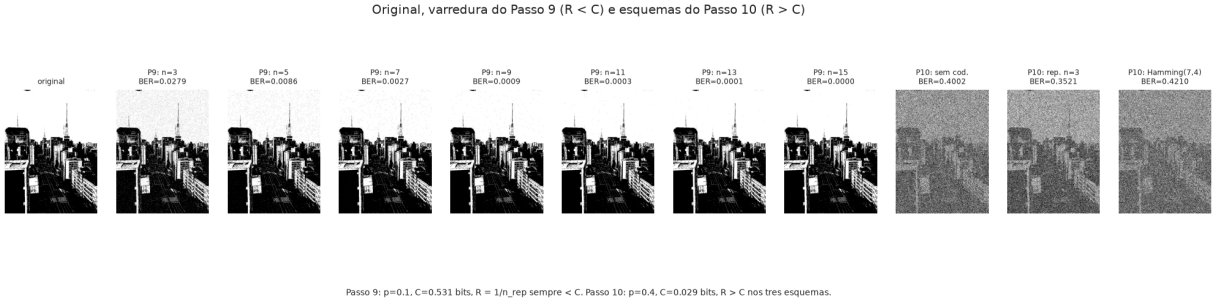
```
        ber_n = np.mean(decod_n != MSG)
        imagens[f"P9: n={n_rep}\nBER={ber_n:.4f}"] = reconstroi(decod_n)

    # os tres esquemas do Passo 10, no canal ruidoso (R > C)
    imagens[f"P10: sem cod.\nBER={ber_direto_alto:.4f}"] =
reconstroi(RECEBIDO_DIRETO_ALTO)
    imagens[f"P10: rep. n={N_REP}\nBER={ber_rep_alto:.4f}"] =
reconstroi(DECOD_REP_ALTO)
    imagens[f"P10: Hamming(7,4)\nBER={ber_ham_alto:.4f}"] =
reconstroi(DECOD_HAM_ALTO)

    fig, eixos = plt.subplots(1, len(imagens), figsize=(2.1 * len(imagens),
4.5))
    for eixo, (titulo, imagem) in zip(eixos, imagens.items()):
        eixo.imshow(imagem[::PASSO, ::PASSO], cmap="gray")
        eixo.set_title(titulo, fontsize=8)
        eixo.axis("off")
    fig.suptitle("Original, varredura do Passo 9 (R < C) e esquemas do
Passo 10 (R > C)")
    fig.text(
        0.5,
        0.02,
        f"Passo 9: p={P_CANAL}, C={C:.3f} bits, R = 1/n_rep sempre < C. "
        f"Passo 10: p={P_CANAL_ALTO}, C={C_ALTO:.3f} bits, R > C nos tres
esquemas.",
        ha="center",
        fontsize=9,
    )
    salvar_fig("t4_passo10_imagens_resultantes")
    plt.show()
else:
    print("sem imagem real disponivel (mensagem sintetica) -- comparacao
visual pulada.")
```

esquema	R	BER	R > C	BER aceitavel
sem codificacao	1.000000	0.400243	True	False
repeticao (n=3)	0.333333	0.352111	True	False
Hamming(7,4)	0.571429	0.420998	True	False

```
p do canal = 0.4, capacidade C = 0.0290 bits
nenhum dos tres esquemas chega perto de BER < 0.01 (todos com R > C)
```



Relatorio

Consolidar resultados, tabelas e figuras em `relatorio.md` (figuras salvas em `figuras/`).